

RESEARCH LIBRARY

Circuit Obstructions

Restoration, complete covers, and the limits of exact projection

Lean and evidence guide

Komorebi P.

Standardized publication edition / 8 October 2026

REALIZATION CALCULUS

Reading scope

Formal definitions, proof dependencies, and the complete declaration register

Legal gates / Shared degree / Complete coverage

The purpose and evidence contract

This companion explains the actual Lean source behind the Circuit Obstructions main paper. It supplies the computational model, the meaning of its predicates, the route from legal gates to the joint lower bound, and all 115 archived theorem types with exact source locations. The written arguments are explained for a mathematical reader; the source proof terms remain authoritative.

The maintained source is local release v1.2.0, Block 02. Its successful formal audit was recorded on 21 September 2026. The present research edition is dated 7 October 2026. These dates identify different operations.

> **Fresh execution and archived formal evidence**
>
> The complete exact Python/C++ verification passed in this edition. A fresh Lean replay was attempted but failed before proof compilation because the `lean` executable is unavailable. It completed no new theorem elaboration, axiom audit, module replay or negative-control rejection. The archived formal records, source hashes and compatible replay instructions are retained.

The formal subset consists of C01, the maintained NCT sharing-degree dependency from F3, and the numerical lower-bound part of C02. C02's evaluation-span dual, affine-input invariance and general monotone ideal, together with C03–C08, remain outside this block. A theorem-name count is not a count of independent discoveries. Forty declarations were inherited and 75 added in Block 02.

Part	Reader's task
Explanatory body	Check the hypotheses and how the predicates represent Boolean mathematics
Proof dependencies	Follow legal reversibility, cut coverage, shared degree, balancing and affine compilation
Evidence chapters	Distinguish written reasoning, exact finite checks, text scans and archived kernel checking
Declaration register	Inspect each complete elaborated type, file, line, block and axiom set

The guide does not supply a new toolchain, an independent checker or independent human review. It preserves the original rights record and the original release files. There is no new publication-novelty or unrestricted hardness claim.

The archived formal endpoint

Let c be a finite legal classical NOT/CNOT/Toffoli circuit. For n input bits and a zero-initialized ancillas, assume for every x and both initial target values b that

$$\mathrm{run}(c)(x, 0^a, b) = (x, 0^a, b \oplus f(x)).$$

Write $T = \mathrm{tc}(c)$ for its number of Toffoli gates. The Block 02 joint endpoint is

Degree and complete-cover bound

$$\boxed{\mathrm{DegreeLE}(T+1, f) \quad \text{and} \quad 2\beta^*(f) \le T+3.}$$

`CircuitObstructions.restored_degree_and_cover_bound`

The first conclusion is an explicit certificate that f has a Boolean polynomial representation of degree at most $T+1$. The second is the affine-optimized complete safe-cover lower bound, equivalently $T \ge \max\{0, 2\beta^*(f) - 3\}$.

This applies even when the output controls intermediate gates, the inputs are modified temporarily, and workspace is dirty internally. It requires restoration only at the boundary. It makes no restriction to clean intermediate representations.

What changed

Block 01 proved that discrepancies between a prefix and an inverse suffix form a complete safe cover. Block 02 supplies the missing degree dependency, proves that the cover parameters exist, constructs the balanced cut, and implements affine-output adjustment using legal zero-Toffoli circuits.

What did not change

The mathematical scope remains the eight maintained claim groups. C02 is still partial as a group: its dual-certificate theorem is not included. The formal source also proves

$$\beta(f) \le \beta^*(f) \le n.$$

This ceiling prevents the degree parameter alone from producing a superpolynomial lower bound. No new best-known bound or publication-novelty claim is made.

Where the proof lives

Every proof term is in `formalization/lean/CircuitObstructions/`. The source inventory, archived 21 September elaboration log, and archived axiom report identify the exact statements. The register in this document lists all 115 maintained theorem declarations, including the 40 inherited declarations. Imported standard-library theorems are not counted.

Model and definitions

THE ASSUMPTIONS ARE PART OF THE THEOREM

A general register state is a Boolean function $W \to \mathrm{Bool}$ on any wire type with decidable equality. C01 does not assume that W is finite. The canonical degree model has a finite wire set with n input wires, a ancillas, and one target. A NOT complements a bit. A CNOT adds one control to a distinct target. A Toffoli adds the product of two distinct controls to a third distinct wire. Gate constructors carry the required inequalities.

All arithmetic is over $\mathbb{F}_2$, implemented using Boolean XOR and AND. Each gate is its own inverse. Circuit reversal, restoration, and every cut are inherited from the unchanged Block 01 source.

Quantity	Meaning in the formal source
T	Toffoli count. NOT and CNOT have cost zero in this count only.
$\mathrm{DegreeLE}(d, f)$	f is an XOR of finitely many monomials, each containing at most d variable occurrences.
Safe detector	A Boolean h with $h(x) = 1 \Rightarrow f(x) = 1$.
Complete coverage	Every positive v has a safe detector with $h(v) = 1$.
$\beta(f)$	Least degree bound sufficient for complete safe coverage.
$\beta^*(f)$	Least upper bound on $\beta(f \oplus g)$ over all degree-one g.

Polynomial certificates are not postulated degree oracles

The inductive certificate has three constructors: zero, a bounded-length monomial, and XOR of two certified functions. It is tied to the concrete evaluator

$$\mathrm{polyEval}(p, x) = \bigoplus_{L \in p} \prod_{i \in L} x_i$$

by a proved equivalence. A certificate yields an explicit finite monomial list, and such a list yields a certificate.

Repeated variables are allowed in the list. Boolean idempotence reduces their repetitions without increasing degree. This is the representation form of the usual Boolean degree bound. The formal source does not separately implement canonical ANF normalization or claim that extracting every homogeneous grade is free.

Boundary of the model

There are no quantum phases, measurements, or discarded work registers. Inputs and zero ancillas restore for every logical input and both target values. Standard classical choice defines least semantic bounds, but supplies no polynomial-time procedure for finding their witnesses.

Why sharing does not break the degree bound

A STRONGER INVARIANT THAN SINGLE-REGISTER DEGREE

For a current state $s(x)$ after t Toffolis, the invariant is

$$\forall L, \qquad \mathrm{DegreeLE}\left(t + |L|,\ x \mapsto \prod_{i \in L} s_i(x)\right).$$

L may be any finite list, including repeated registers. Initially each coordinate is a variable or a constant, so a product of r coordinates has degree at most r.

Suppose the next gate toggles target u by the product of a control list of length q. A product that omits u is unchanged. If u occurs, idempotence removes all its repetitions. With R the remaining list, the updated product is

$$\left(s_u \prod_{i \in R} s_i\right) \oplus \left(\prod_{j \in \mathrm{controls}} s_j \prod_{i \in R} s_i\right).$$

The two old-state products have certified degrees at most

$$t + 1 + |R| \qquad \text{and} \qquad t + q + |R|.$$

Since $|R| + 1 \le |L|$, the invariant's budget rises by at most $\max\{q - 1, 0\}$: zero for NOT and CNOT, one for Toffoli.

Induction along the actual gate list gives

$$\mathrm{DegreeLE}\left(T + r,\ x \mapsto \prod_{i \in L} \mathrm{run}(c)(s(x))_i\right), \qquad r = |L|.$$

With $r = 1$, every register has degree at most $T + 1$.

> **Why this handles a DAG rather than just a formula**
>
> The proof never unfolds a reused intermediate value into duplicated computation. It estimates arbitrary products of the current registers in one shared state. The budget increases once per actual Toffoli, including when its controls share earlier work or its target participates in feedback.

The statements are `workspaceDegree_run` and `canonical_run_coordinate_degree`. This formalizes the maintained NCT specialization of F3. A separate generic XOR–AND circuit syntax and simulation theorem are not added here.

From the cut to a numerical lower bound

EXISTENCE, BALANCE, AND COMPLETE COVERAGE

Write $c = p +\!\!+ q$. At this cut, Block 01 defines

$$H_i(x) = \mathrm{run}(p)(x, 0^a, 0)_i \oplus \mathrm{run}(q^{-1})(x, 0^a, 0)_i$$

and proves

$$H_i \le f, \qquad f(x) = 1 \iff \exists i\ H_i(x) = 1.$$

Reversal does not change Toffoli count. The new coordinate degree theorem therefore gives

$$\deg(H_i) \le \max\{\mathrm{tc}(p), \mathrm{tc}(q)\} + 1.$$

The least safe degree exists

For a positive v, take the point indicator

$$\delta_v(x) = \prod_{i=1}^{n} \begin{cases} x_i, & v_i = 1, \\ 1 + x_i, & v_i = 0. \end{cases}$$

It is safe, detects v, and has a degree-n certificate. Hence complete safe coverage exists at degree n for every function. Strong induction proves the existence of a least admissible natural degree. The definition `beta` chooses this bound and its specification and minimality are proved.

The balanced cut is constructed

Because each gate has cost zero or one, a prefix can realize every Toffoli count from zero to T. The source proves this fact and extracts a cut with

$$\max\{\mathrm{tc}(p), \mathrm{tc}(q)\} \le \lceil T/2 \rceil .$$

The complete safe cut cover then proves

$$\beta(f) \le \lceil T/2 \rceil + 1, \qquad \boxed{2\beta(f) \le T + 3.}$$

Natural-number arithmetic retains the floor/ceiling and small-T cases. No balance assumption is added to the final theorem.

Affine output adjustment is an actual circuit

THE OPTIMIZATION DOES NOT ASSUME A CLEAN NORMAL FORM

For every degree-one Boolean function g, the source constructs a restored implementation with zero Toffolis.

A constant is computed by the empty circuit or target NOT. An input variable uses a legal input-to-target CNOT. XOR is implemented by concatenating restored circuits. Induction on a degree-one certificate gives the general construction.

If c restores f and d restores g, their concatenation restores $f \oplus g$. For the affine implementation,

$$\mathrm{tc}(c +\!+ d) = \mathrm{tc}(c).$$

The unoptimized bound applies to every affine g at the same T. Define $\beta^*(f)$ as the least upper bound of those $\beta(f \oplus g)$ values. Existence follows from their common upper bound n.

Therefore

$$\boxed{2\beta^*(f) \le T + 3, \qquad \beta(f) \le \beta^*(f) \le n.}$$

The final output at initial target zero is exactly f, so the forward coordinate theorem also gives $\mathrm{DegreeLE}(T+1, f)$. The joint endpoint combines these two proved conclusions.

Nonvacuity: two-input AND

Every degree-one function on two inputs has zero XOR over the four corners of the input square. AND has corner XOR one. Thus AND has no degree-one certificate.

A degree-one complete safe cover for AND would include a detector of its sole accepting input. Safety forces that detector to be AND itself, contradicting the corner-parity lemma. The degree-two monomial is a valid detector. The source proves

$$\beta(\mathrm{AND}_2) = \beta^*(\mathrm{AND}_2) = 2$$

and therefore every restored implementation of AND_2 uses at least one Toffoli.

This is a theorem about all implementations in the model, not an exhaustive search through circuits. It does not formalize the general k-input monomial optimum.

Verification and proof boundaries

UNIVERSAL PROOFS AND FINITE CONTROLS REMAIN DISTINCT

Module	Count	Formal role
Gates	16	Inherited legal primitives and reversal
CutComparison	13	Inherited complete cut cover
Examples	11	Inherited dirty and output-active controls
Degree	19	Polynomial certificates and closure
MultiplicativeDegree	12	Degree invariant with sharing
SafeDegree	22	Least complete degree and balanced cuts
AffineOutput	14	Affine compiler and optimized bound
DegreeChecks	8	AND2 exclusion and exact parameters
Total	115	75 new declarations, 40 inherited

The archived 21 September audit rebuilt all source in a fresh project directory, treated warnings as errors, printed each declaration's type and axiom dependencies, and replayed all eight modules through the bundled checker. Its eight deliberately invalid proof controls were rejected. None of these formal operations completed in the 7 October attempt, which stopped because the compiler executable was missing.

Only `propext`, `Classical.choice`, and `Quot.sound` occur in the archived audited dependencies. No custom axiom, admitted proof, native-decision oracle, or kernel bypass is used. Lean 4.34.0-rc2 and its bundled Std are pinned. This checker shares the toolchain's trust boundary and is not an independent implementation or human review.

The separate Python checker, freshly reproduced in this edition, uses exact truth tables and integer Möbius transforms. It covers all 278 functions on zero through three inputs; all words over the documented 12-gate three-wire alphabet through length three and the 28-gate four-wire alphabet through length two (2,698 words); 28,088 register-product checks; and 251 restored words. The universal theorems do not follow by extrapolation from those finite tests.

What remains outside the formal block

C02's evaluation-span dual, affine input invariance, and general monotone ideal are not formalized. C03–C08 retain their previous written and finite-check status. The original classical/reversible circuit-size comparison and general exact monomial-cost theorem are not newly formalized. The active claim register controls interpretation.

Reading the formal model literally

QUANTIFIERS AND ENCODING CARRY THE HYPOTHESES

The generic cut theory uses a logical-input type X, a wire type W with decidable equality, and a boundary structure. Its essential fields are an encoding $\mathrm{encode} : X \to \mathrm{Bool} \to (W \to \mathrm{Bool})$, a target wire, and the proof

$$\mathrm{encode}(x, b)(\mathrm{target}) = b.$$

This target equation separates the two target branches for each fixed input. It excludes an encoding that silently ignores its supplied target bit. The source proves `Boundary.target_separated`; it does not assume an injective encoding of every logical-input value as an additional hypothesis.

The restoration predicate is

$$\forall x\, b, \quad \mathrm{run}(c)(\mathrm{encode}(x, b)) = \mathrm{encode}(x, b + f(x)).$$

It is a hypothesis of the conditional theorem. The code does not assert that every gate list realizes a restored oracle. The canonical encoding sets each input wire to its input bit, each ancillary wire to zero, and the target to b. Its wire type is

```
Sum (Fin n) (Sum (Fin a) Unit)
```

which has exactly $n + a + 1$ wires.

In the generic C01 theorem, completeness is the existential statement

$$f(x) = 1 \quad \Longleftrightarrow \quad \exists i : W,\ H_i(x) = 1.$$

No finite disjunction is needed there. Specializing to the canonical finite wires yields the finite complete family used in the main paper. Completeness does not say that the XOR of all packets reconstructs f; detecting packets can cancel under XOR. It states existence of a detecting coordinate at every positive input.

No clean-intermediate condition occurs in these predicates. Legal gates may temporarily alter inputs, dirty scratch and use the target as a control. The source examples exercise those allowances. Its compute/copy/uncompute AND example uses two Toffolis and is a restoration witness, whereas direct AND uses one Toffoli. Neither example is an unrestricted circuit-search result.

Certificates, semantic minima and natural arithmetic

WHAT THE FORMAL OBJECTS DO AND DO NOT COMPUTE

`DegreeLE d f` is an inductive representation predicate. Its constructors build zero, a monomial represented by a variable-occurrence list of length at most d, and XOR of certified functions. The polynomial evaluator XORs the monomial-list values. The source proves the equivalence between a degree certificate and such a finite representation.

Repeated indices are permitted. Boolean idempotence reduces their repetitions, so the representation gives the usual multilinear degree bound. The formal block supplies no separate canonical ANF normalization algorithm and no theorem that extracting homogeneous grades is computationally free. Zero has a degree-zero certificate. The Python degree helper's -1 convention for zero is a mathematical convention outside the natural-valued formal bound.

`CompleteSafeDegreeLE f d` says that every positive point has some degree-d safe detector. It does not require one detector to cover all positives, and it places no efficiency bound on finding those detectors. Point indicators show that $d = n$ suffices. A least-bound existence theorem then permits the noncomputable definition `beta`. Its specification supplies complete coverage; its minimality compares it with every sufficient natural bound.

`AffineSafeDegreeLE f d` universally quantifies every degree-one output function g and requires $\beta(f + g) \leq d$. The definition `betaStar` chooses its least common upper bound. There are finitely many Boolean functions on n inputs, so this least bound equals the main paper's maximum over affine output masks. It is a maximum over mask values, not a minimum over masks.

The source proves $\beta(f) \leq \beta^*(f) \leq n$. The zero mask supplies the first inequality. The point-indicator bound supplies the second. Classical choice in the definitions is not a polynomial-time algorithm for computing these parameters.

All counters and degree bounds in this block are natural numbers. In `2 * betaStar f - 3`, subtraction saturates at zero. Thus

$$2\beta^*(f) - 3 \leq T$$

in the formal natural-number statement is the integer inequality $\max\{0, 2\beta^*(f) - 3\} \leq T$. The nonsubtractive endpoint $2\beta^*(f) \leq T + 3$ avoids ambiguity. Rounding at odd T is retained; the balanced cut is chosen by Toffoli count, not by half the gate-list length.

The proof dependency route

A SOURCE MAP RATHER THAN A NEW AXIOM

Stage	Source role and conclusion
Legal gates and reversal	Distinct-wire constructor proofs; each gate is self-inverse; reverse gate lists invert the run
Separated boundary	The two initial target branches remain distinct after every prefix
Cut transport	Inverse suffix meets the appropriate forward target branch; $H_i = fD_i$ gives safety and pointwise completeness
Polynomial certificates	Monomial evaluation and XOR/product closure connect concrete lists to degree bounds
Shared register products	Products of r current registers have budget $t + r$ after t Toffolis; repeats and feedback are allowed
Balanced cut	An attainable prefix cost gives both sides cost at most $\lceil T/2 \rceil$; the complete family gives the beta bound
Affine-output circuit	Degree-one certificates compile to restored NOT/CNOT lists, giving zero extra Toffolis for every mask
Joint endpoint	The forward target at initial zero supplies f; degree and optimized-cover conclusions are conjoined

The strongest degree invariant concerns products of actual current registers. It does not duplicate the histories of shared controls. This is why its coordinate consequence applies to DAG reuse, temporary input modification and target feedback in this gate model. The general XOR–AND syntax argument in the main paper remains a separate written proof.

The general run theorem needs degree-one certificates for the initial register functions. The canonical boundary supplies these certificates; restoration is used later to identify the final target with f. The affine existence theorem constructs gate lists inside its proof. It does not expose a separate executable compiler function or assert minimum total gate count.

The balanced cut and least-degree facts are established dependencies. They are not extra assumptions added to `restored_degree_and_cover_bound`. That endpoint assumes a legal gate list and restoration in the canonical boundary model. The register prints its full type as L107.

The evidence ledger for this edition

ARCHIVED KERNEL CHECKING AND FRESH EXACT EXECUTION

The complete archived Block 02 record is `evidence/lean-block02/verified/verification.json` inside the preserved source release. It records 115 theorem declarations, eight replayed modules, and eight rejected deliberately false or forbidden controls. The toolchain is pinned to Lean 4.34.0-rc2 with bundled Std; there is no Mathlib dependency.

Across the archived theorem audit, 25 entries have no axioms; 17 use only `propext`; three use only `Quot.sound`; 40 use those two; and 30 use all three of `propext`, `Classical.choice` and `Quot.sound`. These groups sum to 115. The full register gives each entry's set. No project axiom or admitted proof occurs in the maintained source. The fresh edition's text-level scan supports that syntactic fact but does not re-elaborate proof terms or recompute their axiom dependencies.

The October attempt's record is

```
build/lean/20261007T131428727384Z/verification.json
```

inside the preserved source directory. Its status is failure. The error is a missing `lean` executable before any recorded compiler subprocess. The command list is empty. Consequently the current edition has no new formal declaration count, axiom output, checker replay or rejected-proof control result. The earlier archived success and this failed attempt are both retained.

The current exact scientific replay passed all maintained source checks and the preserved historical runner. Its manifest is

```
build/verification/20261007T131040268104Z-all/manifest.json
```

inside the source directory. The degree-profile checker counts 278 functions on zero through three inputs. The original small-oracle construction suite counts 276 functions on one through three inputs. The two zero-arity functions explain the difference. Those totals are not added together as disjoint proof evidence.

The source seal passed for its 321 sealed source entries and 27 evidence entries. These are integrity counts rather than theorem counts. The publication manifest binds final files separately. PDF compilation, source integrity, finite testing and kernel replay each establish a different kind of fact.

Negative controls and the trust boundary

A REJECTION HAS A PRECISE ROLE

The archived audit deliberately tested the following controls:

Control	Why rejection matters
Illegal repeated wire	Legal gate constructors require distinct controls and target
Target-only cut reconstruction	A discrepancy can move to a data wire; completeness needs the entire coordinate family
False theorem	A false statement is not accepted merely because it has a theorem wrapper
Placeholder guard	The project rejects admitted or placeholder source constructs
False monomial degree	A two-occurrence monomial cannot satisfy the supplied length-at-most-one premise
False affine AND claim	The supplied degree-two proof has the wrong type for a degree-one target
False beta value	Complete safe detection for AND2 cannot have degree one
False betaStar bound	Optimization cannot erase the AND2 obstruction

All eight are archived expected rejections. They help check that the audit would detect the intended errors. They are not eight additional positive mathematical results, an exhaustive attack on every possible checker defect, or controls freshly run in October. The separate positive four-corner theorem proves the semantic impossibility of an affine AND2 certificate; it is not the mechanism of that particular wrong-type negative test.

The archived bundled checker shares the Lean toolchain's trust boundary. It is not an independently implemented proof checker. Historical recovery used a CRC-checked portable archive part, and executable hashes matched the predecessor. The retained historical account does not claim integrity of a complete upstream distribution from that partial stream. A future reader should obtain a complete compatible toolchain through a trusted installation or their own verified source build.

The original development account retains failed download probes, an unstarted streaming launch, incomplete proof attempts and strict-warning errors. The final archived proof terms contain no admitted repair. Those development failures, expected negative controls, archived success and current missing compiler failure remain separate events.

Reproduce and inspect the authoritative files

COMMANDS AND SOURCE IDENTITY

From `source/komorebi-circuit-obstructions-v1.2.0`, run

```
python scripts/check_integrity.py
python scripts/verify.py --all
python scripts/verify_lean.py --toolchain /path/to/toolchain
```

The integrity check needs Python; the full scientific replay also needs a C++17 compiler. The last command needs the pinned Lean toolchain supplied separately. The verification scripts do not download it. A missing toolchain fails before proof checking, as it did in this edition. The compatible command is provided for reproduction, not as a claimed local success.

From the publication-set root, regenerate the exact declaration appendix and the explanatory guide source with

```
python scripts/build_lean_register.py
python scripts/build_lean_guide.py
python scripts/build_documents.py
python scripts/verify_publication.py
```

The register generator verifies all 115 source hashes and declaration lines against the archived catalog, then copies each complete elaborated type. The guide generator preserves the archived explanatory proof route and adds this edition's evidence distinctions. Neither script compiles Lean.

The unmodified source ZIP's SHA-256 is

```
88628c8274660bd20e11fec83249fdc04960403b9d64d1e5068a476b8c025936
```

The complete publication package preserves that ZIP, the repository bundle, original companions and current evidence. Its new manifest binds the revised papers, generators and figure assets. Hash consistency is not an authorship certificate or mathematical proof.

The declaration register follows. It includes the 40 inherited results and 75 Block 02 additions. The ten non-ASCII logical glyphs in its exact type text are rendered as mathematical glyphs for readability. The unchanged UTF-8 sidecar is `manuscripts/lean_statements_utf8.txt`; the original authoritative catalog is `reference/lean-block02/declaration_catalog.json` in the source release. Imported library declarations and explanatory prose are not counted as new project theorems.

Complete declaration register

The following 115 types are transcribed from the archived Block 02 declaration catalog. Unicode logical operators are rendered as mathematical glyphs; the complete original UTF-8 statements are also supplied. Source files and line locators match the preserved source hashes. These are archived elaborated types, not a new compilation result.

Gates: 16 declarations

L001 `write_same`
Source. `CircuitObstructions/Gates.lean:17`
Block. 01 **Axioms.** propext

```
CircuitObstructions.write_same {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) (t : W) (b : Bool) :
  CircuitObstructions.write s t b t = b
```

L002 `write_other`
Source. `CircuitObstructions/Gates.lean:20`
Block. 01 **Axioms.** propext

```
CircuitObstructions.write_other {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) (t i : W) (b : Bool)
  (h : i ≠ t) : CircuitObstructions.write s t b i = s i
```

L003 `xor_cancel_right`
Source. `CircuitObstructions/Gates.lean:23`
Block. 01 **Axioms.** none

```
CircuitObstructions.xor_cancel_right (a b : Bool) : (a ^^ b ^^ b) = a
```

L004 `xor_cancel_left`
Source. `CircuitObstructions/Gates.lean:26`
Block. 01 **Axioms.** none

```
CircuitObstructions.xor_cancel_left (a b : Bool) : (a ^^ (a ^^ b)) = b
```

L005 `xor_commute`
Source. `CircuitObstructions/Gates.lean:29`
Block. 01 **Axioms.** none

```
CircuitObstructions.xor_commute (a b : Bool) : (a ^^ b) = (b ^^ a)
```

L006 `xor_true_iff_ne`
Source. `CircuitObstructions/Gates.lean:32`
Block. 01 **Axioms.** none

```
CircuitObstructions.xor_true_iff_ne (a b : Bool) : (a ^^ b) = true ↔ a ≠ b
```

L007 `eval_involutive`
Source. `CircuitObstructions/Gates.lean:48`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.Gate.eval_involutive {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  (s : CircuitObstructions.State W) : g.eval (g.eval s) = s
```

L008 `eval_injective`
Source. `CircuitObstructions/Gates.lean:67`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.Gate.eval_injective {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  {s t : CircuitObstructions.State W} (h : g.eval s = g.eval t) : s = t
```

L009 `run_nil`
Source. `CircuitObstructions/Gates.lean:78`
Block. 01 **Axioms.** none

```
CircuitObstructions.run_nil {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) :
  CircuitObstructions.run [] s = s
```

L010 `run_cons`
Source. `CircuitObstructions/Gates.lean:81`
Block. 01 **Axioms.** none

```
CircuitObstructions.run_cons {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  (gs : CircuitObstructions.Circuit W) (s : CircuitObstructions.State W) :
  CircuitObstructions.run (g :: gs) s = CircuitObstructions.run gs (g.eval s)
```

L011 `run_append`
Source. `CircuitObstructions/Gates.lean:84`
Block. 01 **Axioms.** none

```
CircuitObstructions.run_append {W : Type} [DecidableEq W] (p q : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) :
  CircuitObstructions.run (p ++ q) s = CircuitObstructions.run q (CircuitObstructions.run p s)
```

L012 `run_reverse_run`
Source. `CircuitObstructions/Gates.lean:90`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.run_reverse_run {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) : CircuitObstructions.run (List.reverse c) (CircuitObstructions.run c s) = s
```

L013 `run_run_reverse`
Source. `CircuitObstructions/Gates.lean:98`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.run_run_reverse {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) : CircuitObstructions.run c (CircuitObstructions.run (List.reverse c) s) = s
```

L014 `run_injective`
Source. `CircuitObstructions/Gates.lean:102`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.run_injective {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  {s t : CircuitObstructions.State W} (h : CircuitObstructions.run c s = CircuitObstructions.run c t) : s = t
```

L015 `toffoliCount_append`
Source. `CircuitObstructions/Gates.lean:114`
Block. 01 **Axioms.** propext

```
CircuitObstructions.toffoliCount_append {W : Type} (p q : CircuitObstructions.Circuit W) :
  CircuitObstructions.toffoliCount (p ++ q) = CircuitObstructions.toffoliCount p + CircuitObstructions.toffoliCount
    q
```

L016 `toffoliCount_reverse`
Source. `CircuitObstructions/Gates.lean:118`
Block. 01 **Axioms.** propext

```
CircuitObstructions.toffoliCount_reverse {W : Type} (c : CircuitObstructions.Circuit W) :
  CircuitObstructions.toffoliCount (List.reverse c) = CircuitObstructions.toffoliCount c
```

CutComparison: 13 declarations

L017 `target_separated`
Source. `CircuitObstructions/CutComparison.lean:17`
Block. 01 **Axioms.** propext

```
CircuitObstructions.Boundary.target_separated {X W : Type} (e : CircuitObstructions.Boundary X W) (x : X) :
  e.encode x false ≠ e.encode x true
```

L018 `canonical_input`
Source. `CircuitObstructions/CutComparison.lean:39`
Block. 01 **Axioms.** none

```
CircuitObstructions.canonical_input (n a : Nat) (x : Fin n → Bool) (b : Bool) (i : Fin n) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inl i) = x i
```

L019 `canonical_ancilla`
Source. `CircuitObstructions/CutComparison.lean:42`
Block. 01 **Axioms.** none

```
CircuitObstructions.canonical_ancilla (n a : Nat) (x : Fin n → Bool) (b : Bool) (i : Fin a) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inr (Sum.inl i)) = false
```

L020 `canonical_target`
Source. `CircuitObstructions/CutComparison.lean:45`
Block. 01 **Axioms.** none

```
CircuitObstructions.canonical_target (n a : Nat) (x : Fin n → Bool) (b : Bool) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inr (Sum.inr ())) = b
```

L021 `inverse_suffix_boundary`
Source. `CircuitObstructions/CutComparison.lean:49`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.inverse_suffix_boundary {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (b : Bool) :
  CircuitObstructions.run (List.reverse q) (e.encode x b) = CircuitObstructions.run p (e.encode x (b ^^ f x))
```

L022 `direction_nonzero`
Source. `CircuitObstructions/CutComparison.lean:67`
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.direction_nonzero {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p : CircuitObstructions.Circuit W) (x : X) : ∃ i, CircuitObstructions.direction e p x i = true
```

L023 `cut_transport_identity`
Source. `CircuitObstructions/CutComparison.lean:82`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.cut_transport_identity {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (i : W) : CircuitObstructions.packet e p q x i = (f x && CircuitObstructions.direction e p x i)
```

L024 `cut_packet_safe`
Source. `CircuitObstructions/CutComparison.lean:91`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.cut_packet_safe {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (i : W) : CircuitObstructions.packet e p q x i = true → f x = true
```

L025 `cut_cover_complete`
Source. `CircuitObstructions/CutComparison.lean:99`
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.cut_cover_complete {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X) :
  f x = true ↔ ∃ i, CircuitObstructions.packet e p q x i = true
```

L026 `complete_safe_cover`
Source. `CircuitObstructions/CutComparison.lean:111`
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.complete_safe_cover {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) :
  (∀ (x : X) (i : W), CircuitObstructions.packet e p q x i = true → f x = true) ∧
    ∀ (x : X), f x = true ↔ ∃ i, CircuitObstructions.packet e p q x i = true
```

L027 `cut_states_equal_iff_reject`
Source. `CircuitObstructions/CutComparison.lean:119`
Block. 01 **Axioms.** propext, Quot.sound

```
CircuitObstructions.cut_states_equal_iff_reject {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X) :
  CircuitObstructions.run p (e.encode x false) = CircuitObstructions.run (List.reverse q) (e.encode x false) ↔
    f x = false
```

L028 `every_index_cut`
Source. `CircuitObstructions/CutComparison.lean:131`
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.every_index_cut {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (c : CircuitObstructions.Circuit W) (f : X → Bool) (hR : CircuitObstructions.Restores e c f) (k : Nat) (x : X) :
  f x = true ↔ ∃ i, CircuitObstructions.packet e (List.take k c) (List.drop k c) x i = true
```

L029 canonical_complete_safe_cover
Source. CircuitObstructions/CutComparison.lean:139
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.canonical_complete_safe_cover (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : (Fin n → Bool) → Bool)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) (k : Nat) :
  (∀ (x : Fin n → Bool) (i : CircuitObstructions.CanonicalWire n a),
      CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) (List.take k c) (List.drop k c) x i =
          true →
        f x = true) ∧
    ∀ (x : Fin n → Bool),
      f x = true ↔
        ∃ i,
          CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) (List.take k c) (List.drop k c) x i
      =
            true
```

Examples: 11 declarations

L030 andOracle_restores
Source. CircuitObstructions/Examples.lean:27
Block. 01 **Axioms.** Quot.sound

```
CircuitObstructions.Examples.andOracle_restores :
  CircuitObstructions.Restores CircuitObstructions.Examples.andBoundary CircuitObstructions.Examples.andOracle fun x
      =>
    x.fst && x.snd
```

L031 andOracle_toffoli_count
Source. CircuitObstructions/Examples.lean:33
Block. 01 **Axioms.** propext

```
CircuitObstructions.Examples.andOracle_toffoli_count :
  CircuitObstructions.toffoliCount CircuitObstructions.Examples.andOracle = 2
```

L032 and_scratch_can_be_dirty
Source. CircuitObstructions/Examples.lean:36
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.and_scratch_can_be_dirty :
  CircuitObstructions.run CircuitObstructions.Examples.andCompute
      (CircuitObstructions.Examples.andBoundary.encode (true, true) false)
      CircuitObstructions.Examples.WorkWire.scratch =
    true
```

L033 andOracle_every_cut
Source. CircuitObstructions/Examples.lean:40
Block. 01 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.Examples.andOracle_every_cut (k : Nat) (x : Bool × Bool) :
  (x.fst && x.snd) = true ↔
    ∃ i,
      CircuitObstructions.packet CircuitObstructions.Examples.andBoundary
          (List.take k CircuitObstructions.Examples.andOracle) (List.drop k CircuitObstructions.Examples.andOracle)
    x
          i =
        true
```

L034 outputActiveOracle_restores
Source. CircuitObstructions/Examples.lean:66
Block. 01 **Axioms.** Quot.sound

```
CircuitObstructions.Examples.outputActiveOracle_restores :
  CircuitObstructions.Restores CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.
     outputActiveOracle
    fun x => true
```

L035 outputActiveOracle_length
Source. CircuitObstructions/Examples.lean:71
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.outputActiveOracle_length : List.length CircuitObstructions.Examples.outputActiveOracle
     = 7
```

L036 outputActiveOracle_toffoli_count
Source. CircuitObstructions/Examples.lean:73
Block. 01 **Axioms.** propext

```
CircuitObstructions.Examples.outputActiveOracle_toffoli_count :
  CircuitObstructions.toffoliCount CircuitObstructions.Examples.outputActiveOracle = 0
```

L037 moved_discrepancy_data
Source. CircuitObstructions/Examples.lean:76
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.moved_discrepancy_data (x : Bool) :
  CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
      CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.data =
    true
```

L038 moved_discrepancy_target
Source. CircuitObstructions/Examples.lean:80
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.moved_discrepancy_target (x : Bool) :
  CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
      CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.target =
    false
```

L039 target_only_reconstruction_is_false
Source. CircuitObstructions/Examples.lean:85
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.target_only_reconstruction_is_false :
  ¬∀ (x : Bool),
      CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
          CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.target =
        true
```

L040 degenerate_encoding_control
Source. CircuitObstructions/Examples.lean:93
Block. 01 **Axioms.** none

```
CircuitObstructions.Examples.degenerate_encoding_control :
  have bad := fun x x_1 => false;
  (∀ (y : Bool), CircuitObstructions.run [] (bad y) = bad (y ^^ true)) ∧
    ∀ (i : CircuitObstructions.Examples.SwapWire), (bad false i ^^ bad false i) = false
```

Degree: 19 declarations

L041 `bprod_nil`
Source. `CircuitObstructions/Degree.lean:19`
Block. 02 **Axioms.** none

```
CircuitObstructions.bprod_nil {W : Type} (s : CircuitObstructions.State W) : CircuitObstructions.bprod [] s = true
```

L042 `bprod_cons`
Source. `CircuitObstructions/Degree.lean:20`
Block. 02 **Axioms.** none

```
CircuitObstructions.bprod_cons {W : Type} (i : W) (is : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod (i :: is) s = (s i && CircuitObstructions.bprod is s)
```

L043 `bprod_append`
Source. `CircuitObstructions/Degree.lean:23`
Block. 02 **Axioms.** propext

```
CircuitObstructions.bprod_append {W : Type} (l k : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod (l ++ k) s = (CircuitObstructions.bprod l s && CircuitObstructions.bprod k s)
```

L044 `xor_and_right`
Source. `CircuitObstructions/Degree.lean:29`
Block. 02 **Axioms.** none

```
CircuitObstructions.xor_and_right (a b c : Bool) : ((a ^^ b) && c) = (a && c ^^ b && c)
```

L045 `and_xor_right`
Source. `CircuitObstructions/Degree.lean:33`
Block. 02 **Axioms.** none

```
CircuitObstructions.and_xor_right (a b c : Bool) : (a && (b ^^ c)) = (a && b ^^ a && c)
```

L046 `not_as_xor_one`
Source. `CircuitObstructions/Degree.lean:37`
Block. 02 **Axioms.** none

```
CircuitObstructions.not_as_xor_one (a : Bool) : (!a) = (a ^^ true)
```

L047 `mono`
Source. `CircuitObstructions/Degree.lean:47`
Block. 02 **Axioms.** none

```
CircuitObstructions.DegreeLE.mono {n d e : Nat} {f : CircuitObstructions.BFun n} (h : CircuitObstructions.DegreeLE d
      f)
  (hde : d ≤ e) : CircuitObstructions.DegreeLE e f
```

L048 `one`
Source. `CircuitObstructions/Degree.lean:54`
Block. 02 **Axioms.** none

```
CircuitObstructions.DegreeLE.one (n d : Nat) : CircuitObstructions.DegreeLE d fun x => true
```

L049 `constant`
Source. `CircuitObstructions/Degree.lean:57`
Block. 02 **Axioms.** none

```
CircuitObstructions.DegreeLE.constant (n d : Nat) (b : Bool) : CircuitObstructions.DegreeLE d fun x => b
```

L050 `variable`
Source. `CircuitObstructions/Degree.lean:63`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.DegreeLE.variable {n : Nat} (i : Fin n) : CircuitObstructions.DegreeLE 1 fun x => x i
```

L051 `xor`
Source. `CircuitObstructions/Degree.lean:67`
Block. 02 **Axioms.** propext

```
CircuitObstructions.DegreeLE.xor {n d e : Nat} {f g : CircuitObstructions.BFun n}
  (hf : CircuitObstructions.DegreeLE d f) (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (max d e) fun x => f x ^^ g x
```

L052 `not`
Source. `CircuitObstructions/Degree.lean:72`
Block. 02 **Axioms.** Quot.sound

```
CircuitObstructions.DegreeLE.not {n d : Nat} {f : CircuitObstructions.BFun n} (hf : CircuitObstructions.DegreeLE d f
    ) :
  CircuitObstructions.DegreeLE d fun x => !f x
```

L053 `monomial_mul`
Source. `CircuitObstructions/Degree.lean:76`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.DegreeLE.monomial_mul {n d e : Nat} (l : List (Fin n)) (hl : l.length ≤ d)
  {g : CircuitObstructions.BFun n} (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (d + e) fun x => CircuitObstructions.bprod l x && g x
```

L054 `mul`
Source. `CircuitObstructions/Degree.lean:87`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.DegreeLE.mul {n d e : Nat} {f g : CircuitObstructions.BFun n}
  (hf : CircuitObstructions.DegreeLE d f) (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (d + e) fun x => f x && g x
```

L055 `product`
Source. `CircuitObstructions/Degree.lean:96`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.DegreeLE.product {n : Nat} {W : Type} (l : List W)
  (s : (Fin n → Bool) → CircuitObstructions.State W) (h : ∀ (i : W), CircuitObstructions.DegreeLE 1 fun x => s x i
    ) :
  CircuitObstructions.DegreeLE l.length fun x => CircuitObstructions.bprod l (s x)
```

L056 `polyEval_append`
Source. `CircuitObstructions/Degree.lean:109`
Block. 02 **Axioms.** propext

```
CircuitObstructions.polyEval_append {n : Nat} (p q : List (List (Fin n))) (x : Fin n → Bool) :
  CircuitObstructions.polyEval (p ++ q) x = (CircuitObstructions.polyEval p x ^^ CircuitObstructions.polyEval q x)
```

L057 `polynomial_witness`
Source. `CircuitObstructions/Degree.lean:115`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.DegreeLE.polynomial_witness {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.DegreeLE d f) :
  ∃ p, (∀ (l : List (Fin n)), l ∈ p → l.length ≤ d) ∧ CircuitObstructions.polyEval p = f
```

L058 `polynomial_degree_certificate`
Source. `CircuitObstructions/Degree.lean:134`
Block. 02 **Axioms.** propext

```
CircuitObstructions.polynomial_degree_certificate {n d : Nat} (p : List (List (Fin n)))
  (hp : ∀ (l : List (Fin n)), l ∈ p → l.length ≤ d) : CircuitObstructions.DegreeLE d (CircuitObstructions.polyEval
     p)
```

L059 `degreeLE_iff_polynomial`
Source. `CircuitObstructions/Degree.lean:142`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.degreeLE_iff_polynomial {n d : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.DegreeLE d f ↔
    ∃ p, (∀ (l : List (Fin n)), l ∈ p → l.length ≤ d) ∧ CircuitObstructions.polyEval p = f
```

MultiplicativeDegree: 12 declarations

L060 `withoutWire_length_le`
Source. `CircuitObstructions/MultiplicativeDegree.lean:17`
Block. 02 **Axioms.** propext

```
CircuitObstructions.withoutWire_length_le {W : Type} [DecidableEq W] (l : List W) (t : W) :
  (CircuitObstructions.withoutWire l t).length ≤ l.length
```

L061 `withoutWire_length_lt`
Source. `CircuitObstructions/MultiplicativeDegree.lean:21`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.withoutWire_length_lt {W : Type} [DecidableEq W] (l : List W) (t : W) (ht : t ∈ l) :
  (CircuitObstructions.withoutWire l t).length + 1 ≤ l.length
```

L062 `bprod_write`
Source. `CircuitObstructions/MultiplicativeDegree.lean:33`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.bprod_write {W : Type} [DecidableEq W] (l : List W) (s : CircuitObstructions.State W) (t : W)
  (b : Bool) :
  CircuitObstructions.bprod l (CircuitObstructions.write s t b) =
    ((if t ∈ l then b else true) && CircuitObstructions.bprod (CircuitObstructions.withoutWire l t) s)
```

L063 `bprod_write_absent`
Source. `CircuitObstructions/MultiplicativeDegree.lean:48`
Block. 02 **Axioms.** propext

```
CircuitObstructions.bprod_write_absent {W : Type} [DecidableEq W] (l : List W) (s : CircuitObstructions.State W) (t :
     W)
  (b : Bool) (ht : ¬t ∈ l) :
  CircuitObstructions.bprod l (CircuitObstructions.write s t b) = CircuitObstructions.bprod l s
```

L064 workspaceDegree_initial

Source. CircuitObstructions/MultiplicativeDegree.lean:62
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.workspaceDegree_initial {n : Nat} {W : Type} (s : (Fin n → Bool) → CircuitObstructions.State W
    )
  (h : ∀ (i : W), CircuitObstructions.DegreeLE 1 fun x => s x i) : CircuitObstructions.WorkspaceDegree s 0
```

L065 workspaceDegree_toggle

Source. CircuitObstructions/MultiplicativeDegree.lean:73
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.workspaceDegree_toggle {n : Nat} {W : Type} [DecidableEq W]
  (s : (Fin n → Bool) → CircuitObstructions.State W) (t : W) (controls : List W) (k : Nat)
  (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (CircuitObstructions.toggleProduct s t controls) (k + (controls.length - 1))
```

L066 workspaceDegree_gate

Source. CircuitObstructions/MultiplicativeDegree.lean:104
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.workspaceDegree_gate {n : Nat} {W : Type} [DecidableEq W]
  (s : (Fin n → Bool) → CircuitObstructions.State W) (g : CircuitObstructions.Gate W) (k : Nat)
  (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (fun x => g.eval (s x)) (k + g.toffoliCost)
```

L067 workspaceDegree_run

Source. CircuitObstructions/MultiplicativeDegree.lean:128
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.workspaceDegree_run {n : Nat} {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : (Fin n → Bool) → CircuitObstructions.State W) (k : Nat) (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (fun x => CircuitObstructions.run c (s x))
    (k + CircuitObstructions.toffoliCount c)
```

L068 run_coordinate_degree

Source. CircuitObstructions/MultiplicativeDegree.lean:139
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.run_coordinate_degree {n : Nat} {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : (Fin n → Bool) → CircuitObstructions.State W) (hs : ∀ (i : W), CircuitObstructions.DegreeLE 1 fun x => s x
    i)
  (i : W) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) fun x => CircuitObstructions.run c (s x) i
```

L069 canonical_boundary_degree

Source. CircuitObstructions/MultiplicativeDegree.lean:146
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_boundary_degree (n a : Nat) (b : Bool) (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE 1 fun x => (CircuitObstructions.canonicalBoundary n a).encode x b i
```

L070 canonical_run_coordinate_degree
Source. CircuitObstructions/MultiplicativeDegree.lean:155
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_run_coordinate_degree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (b : Bool)
  (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) fun x =>
    CircuitObstructions.run c ((CircuitObstructions.canonicalBoundary n a).encode x b) i
```

L071 canonical_cut_packet_degree
Source. CircuitObstructions/MultiplicativeDegree.lean:161
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_cut_packet_degree (n a : Nat)
  (p q : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a))
  (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE (max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) + 1)
    fun x => CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) p q x i
```

SafeDegree: 22 declarations

L072 mono
Source. CircuitObstructions/SafeDegree.lean:16
Block. 02 **Axioms.** none

```
CircuitObstructions.CompleteSafeDegreeLE.mono {n d e : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.CompleteSafeDegreeLE f d) (hde : d ≤ e) : CircuitObstructions.CompleteSafeDegreeLE f e
```

L073 bprod_true_iff
Source. CircuitObstructions/SafeDegree.lean:22
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.bprod_true_iff {W : Type} (l : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod l s = true ↔ ∀ (i : W), i ∈ l → s i = true
```

L074 pointIndicator_true_iff
Source. CircuitObstructions/SafeDegree.lean:31
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.pointIndicator_true_iff {n : Nat} (v x : Fin n → Bool) :
  CircuitObstructions.pointIndicator v x = true ↔ x = v
```

L075 pointIndicator_degree
Source. CircuitObstructions/SafeDegree.lean:44
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.pointIndicator_degree {n : Nat} (v : Fin n → Bool) :
  CircuitObstructions.DegreeLE n (CircuitObstructions.pointIndicator v)
```

L076 completeSafeDegree_le_inputs
Source. CircuitObstructions/SafeDegree.lean:56
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.completeSafeDegree_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.CompleteSafeDegreeLE f n
```

L077 `exists_least_degree`
Source. `CircuitObstructions/SafeDegree.lean:65`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.exists_least_degree (P : Nat → Prop) (n : Nat) (hn : P n) : ∃ r, P r ∧ ∀ (k : Nat), P k → r ≤
    k
```

L078 `beta_exists`
Source. `CircuitObstructions/SafeDegree.lean:83`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_exists {n : Nat} (f : CircuitObstructions.BFun n) :
  ∃ r, CircuitObstructions.CompleteSafeDegreeLE f r ∧ ∀ (k : Nat), CircuitObstructions.CompleteSafeDegreeLE f k → r
    ≤ k
```

L079 `beta_spec`
Source. `CircuitObstructions/SafeDegree.lean:90`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_spec {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.CompleteSafeDegreeLE f (CircuitObstructions.beta f)
```

L080 `beta_le_of_cover`
Source. `CircuitObstructions/SafeDegree.lean:93`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_le_of_cover {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.CompleteSafeDegreeLE f d) : CircuitObstructions.beta f ≤ d
```

L081 `beta_le_iff`
Source. `CircuitObstructions/SafeDegree.lean:97`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_le_iff {n d : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.beta f ≤ d ↔ CircuitObstructions.CompleteSafeDegreeLE f d
```

L082 `beta_le_inputs`
Source. `CircuitObstructions/SafeDegree.lean:103`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) : CircuitObstructions.beta f ≤ n
```

L083 `beta_zero`
Source. `CircuitObstructions/SafeDegree.lean:106`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_zero (n : Nat) : (CircuitObstructions.beta fun x => false) = 0
```

L084 `canonical_cut_completeSafeDegree`
Source. `CircuitObstructions/SafeDegree.lean:113`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.canonical_cut_completeSafeDegree (n a : Nat)
  (p q : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) (p ++ q) f) :
  CircuitObstructions.CompleteSafeDegreeLE f
    (max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) + 1)
```

L085 `toffoliCost_le_one`
Source. `CircuitObstructions/SafeDegree.lean:124`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.Gate.toffoliCost_le_one {W : Type} (g : CircuitObstructions.Gate W) : g.toffoliCost ≤ 1
```

L086 `toffoliCount_nil`
Source. `CircuitObstructions/SafeDegree.lean:127`
Block. 02 **Axioms.** propext

```
CircuitObstructions.toffoliCount_nil {W : Type} : CircuitObstructions.toffoliCount [] = 0
```

L087 `toffoliCount_cons`
Source. `CircuitObstructions/SafeDegree.lean:129`
Block. 02 **Axioms.** propext

```
CircuitObstructions.toffoliCount_cons {W : Type} (g : CircuitObstructions.Gate W) (c : CircuitObstructions.Circuit W
    ) :
  CircuitObstructions.toffoliCount (g :: c) = g.toffoliCost + CircuitObstructions.toffoliCount c
```

L088 `exists_cut_with_toffoliCount`
Source. `CircuitObstructions/SafeDegree.lean:133`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.exists_cut_with_toffoliCount {W : Type} (c : CircuitObstructions.Circuit W) (k : Nat)
  (hk : k ≤ CircuitObstructions.toffoliCount c) : ∃ p q, c = p ++ q ∧ CircuitObstructions.toffoliCount p = k
```

L089 `exists_balanced_toffoli_cut`
Source. `CircuitObstructions/SafeDegree.lean:155`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.exists_balanced_toffoli_cut {W : Type} (c : CircuitObstructions.Circuit W) :
  ∃ p q,
    c = p ++ q ∧
      max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) ≤
        (CircuitObstructions.toffoliCount c + 1) / 2
```

L090 `canonical_restored_completeSafeDegree`
Source. `CircuitObstructions/SafeDegree.lean:165`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.canonical_restored_completeSafeDegree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.CompleteSafeDegreeLE f ((CircuitObstructions.toffoliCount c + 1) / 2 + 1)
```

L091 `beta_le_half_toffoliCount`
Source. `CircuitObstructions/SafeDegree.lean:173`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_le_half_toffoliCount (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.beta f ≤ (CircuitObstructions.toffoliCount c + 1) / 2 + 1
```

L092 `beta_restoration_bound`
Source. `CircuitObstructions/SafeDegree.lean:179`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_restoration_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.beta f ≤ CircuitObstructions.toffoliCount c + 3
```

L093 `toffoliCount_ge_two_beta_sub_three`
Source. `CircuitObstructions/SafeDegree.lean:186`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.toffoliCount_ge_two_beta_sub_three (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.beta f - 3 ≤ CircuitObstructions.toffoliCount c
```

AffineOutput: 14 declarations

L094 `restores_append`
Source. `CircuitObstructions/AffineOutput.lean:12`
Block. 02 **Axioms.** propext

```
CircuitObstructions.restores_append {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (c d : CircuitObstructions.Circuit W) (f g : X → Bool) (hc : CircuitObstructions.Restores e c f)
  (hd : CircuitObstructions.Restores e d g) : CircuitObstructions.Restores e (c ++ d) fun x => f x ^^ g x
```

L095 `canonical_target_not`
Source. `CircuitObstructions/AffineOutput.lean:20`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_target_not (n a : Nat) (x : Fin n → Bool) (b : Bool) :
  (CircuitObstructions.Gate.not (CircuitObstructions.canonicalBoundary n a).target).eval
      ((CircuitObstructions.canonicalBoundary n a).encode x b) =
    (CircuitObstructions.canonicalBoundary n a).encode x (b ^^ true)
```

L096 `canonical_input_to_target`
Source. `CircuitObstructions/AffineOutput.lean:34`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_input_to_target (n a : Nat) (i : Fin n) (x : Fin n → Bool) (b : Bool) :
  (CircuitObstructions.inputToTarget n a i).eval ((CircuitObstructions.canonicalBoundary n a).encode x b) =
    (CircuitObstructions.canonicalBoundary n a).encode x (b ^^ x i)
```

L097 `affine_restored_exists`
Source. `CircuitObstructions/AffineOutput.lean:45`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.affine_restored_exists (n a : Nat) (g : CircuitObstructions.BFun n)
  (hg : CircuitObstructions.DegreeLE 1 g) :
  ∃ d,
    CircuitObstructions.toffoliCount d = 0 ∧
      CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) d g
```

L098 `betaStar_exists`
Source. `CircuitObstructions/AffineOutput.lean:76`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_exists {n : Nat} (f : CircuitObstructions.BFun n) :
  ∃ r, CircuitObstructions.AffineSafeDegreeLE f r ∧ ∀ (k : Nat), CircuitObstructions.AffineSafeDegreeLE f k → r ≤
    k
```

L099 `betaStar_spec`
Source. `CircuitObstructions/AffineOutput.lean:85`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_spec {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.AffineSafeDegreeLE f (CircuitObstructions.betaStar f)
```

L100 `betaStar_le_of_bound`
Source. `CircuitObstructions/AffineOutput.lean:88`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_le_of_bound {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.AffineSafeDegreeLE f d) : CircuitObstructions.betaStar f ≤ d
```

L101 `betaStar_le_inputs`
Source. `CircuitObstructions/AffineOutput.lean:92`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) : CircuitObstructions.betaStar f ≤
    n
```

L102 `beta_le_betaStar`
Source. `CircuitObstructions/AffineOutput.lean:97`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_le_betaStar {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.beta f ≤ CircuitObstructions.betaStar f
```

L103 `betaStar_le_half_toffoliCount`
Source. `CircuitObstructions/AffineOutput.lean:101`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_le_half_toffoliCount (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.betaStar f ≤ (CircuitObstructions.toffoliCount c + 1) / 2 + 1
```

L104 `betaStar_restoration_bound`
Source. `CircuitObstructions/AffineOutput.lean:112`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_restoration_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.betaStar f ≤ CircuitObstructions.toffoliCount c + 3
```

L105 `toffoliCount_ge_two_betaStar_sub_three`
Source. `CircuitObstructions/AffineOutput.lean:119`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.toffoliCount_ge_two_betaStar_sub_three (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.betaStar f - 3 ≤ CircuitObstructions.toffoliCount c
```

L106 `canonical_restored_output_degree`
Source. `CircuitObstructions/AffineOutput.lean:126`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.canonical_restored_output_degree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) f
```

L107 `restored_degree_and_cover_bound`
Source. `CircuitObstructions/AffineOutput.lean:140`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.restored_degree_and_cover_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) f ∧
    2 * CircuitObstructions.betaStar f ≤ CircuitObstructions.toffoliCount c + 3
```

DegreeChecks: 8 declarations

L108 `squareParity_add`
Source. `CircuitObstructions/DegreeChecks.lean:17`
Block. 02 **Axioms.** propext

```
CircuitObstructions.squareParity_add (f g : CircuitObstructions.BFun 2) :
  (CircuitObstructions.squareParity fun x => f x ^^ g x) =
    (CircuitObstructions.squareParity f ^^ CircuitObstructions.squareParity g)
```

L109 `affine_squareParity_zero`
Source. `CircuitObstructions/DegreeChecks.lean:22`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.affine_squareParity_zero (f : CircuitObstructions.BFun 2) (h : CircuitObstructions.DegreeLE 1 f)
      :
  CircuitObstructions.squareParity f = false
```

L110 `andTwo_degree_two`
Source. `CircuitObstructions/DegreeChecks.lean:39`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.andTwo_degree_two : CircuitObstructions.DegreeLE 2 CircuitObstructions.andTwo
```

L111 `andTwo_not_affine`
Source. `CircuitObstructions/DegreeChecks.lean:43`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.andTwo_not_affine : ¬CircuitObstructions.DegreeLE 1 CircuitObstructions.andTwo
```

L112 `andTwo_no_degree_one_safe_cover`
Source. `CircuitObstructions/DegreeChecks.lean:49`
Block. 02 **Axioms.** propext, Quot.sound

```
CircuitObstructions.andTwo_no_degree_one_safe_cover :
  ¬CircuitObstructions.CompleteSafeDegreeLE CircuitObstructions.andTwo 1
```

L113 `beta_andTwo`
Source. `CircuitObstructions/DegreeChecks.lean:64`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.beta_andTwo : CircuitObstructions.beta CircuitObstructions.andTwo = 2
```

L114 `betaStar_andTwo`
Source. `CircuitObstructions/DegreeChecks.lean:72`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.betaStar_andTwo : CircuitObstructions.betaStar CircuitObstructions.andTwo = 2
```

L115 `restored_andTwo_needs_toffoli`
Source. `CircuitObstructions/DegreeChecks.lean:78`
Block. 02 **Axioms.** propext, Classical.choice, Quot.sound

```
CircuitObstructions.restored_andTwo_needs_toffoli (a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire 2 a))
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary 2 a) c CircuitObstructions.andTwo) :
  1 ≤ CircuitObstructions.toffoliCount c
```